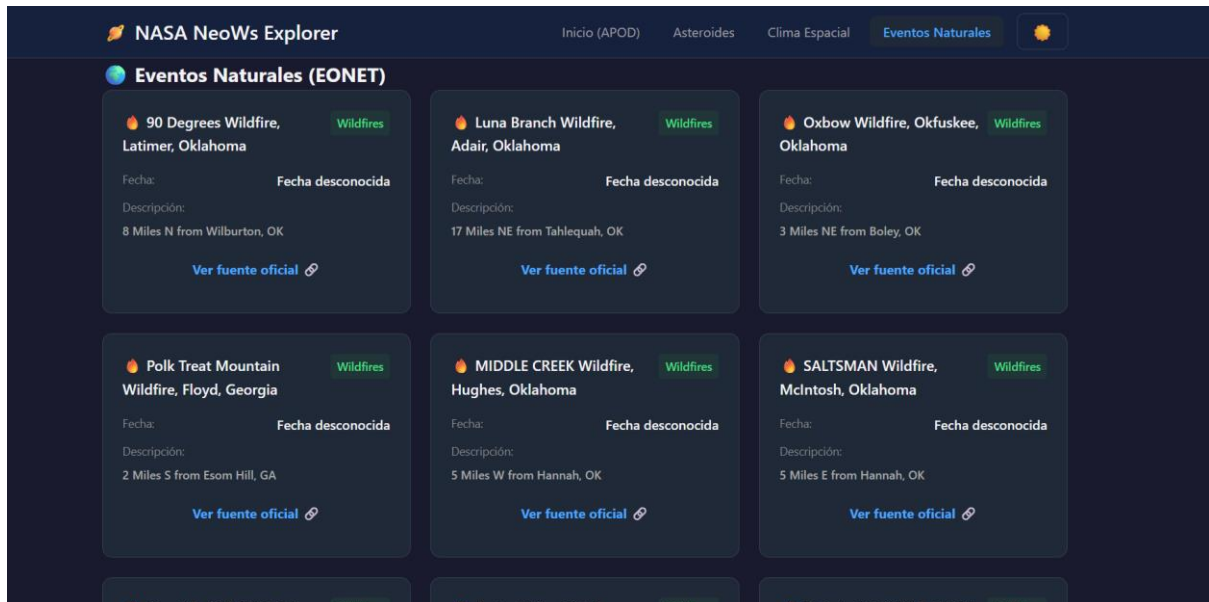


NASA NeoWs Explorer v2



Grupo: Miguel Ruiz Merino, Rubén Pérez Gonzalez

1. Introducción y Objetivos

En esta primera parte del proyecto hemos desarrollado una aplicación web funcional utilizando HTML, CSS y JavaScript (Vanilla), trabajando sin un backend propio y consumiendo datos en tiempo real desde diversas APIs públicas de la NASA.

Sin embargo, el objetivo principal de este proyecto no era solo obtener un resultado visual atractivo, sino aprender e interiorizar la filosofía de desarrollo mediante Sistemas Multi-Agente (utilizando la extensión de Gemini en VS Code como nuestro entorno de agentes).

Nuestros objetivos clave han sido:

Construir una aplicación web funcional y modular cumpliendo los requisitos técnicos exigidos (Principios SOLID, sin eventos inline, responsive, etc.).

Dirigir el desarrollo utilizando agentes de IA, asumiendo nosotros el rol de orquestadores.

Minimizar la intervención manual directa en el código, apoyándonos en la generación iterativa mediante prompts estructurados y asignación de roles a la IA.

Documentar minuciosamente las decisiones, el flujo de comunicación y la resolución de incidencias.

La temática elegida ha sido la exploración espacial, ya que nos permitía conectar múltiples endpoints complejos (imágenes, asteroides, clima espacial) y presentarlos en un Dashboard unificado.

2. Metodología de Trabajo con Agentes (El Proceso)

Para llevar a cabo el proyecto, decidimos no programar de forma tradicional. En su lugar, utilizamos el modelo de IA a través de la extensión de nuestro editor (VS Code), dividiendo el trabajo mediante la asignación de Roles Específicos (Agentes).

Definimos un esquema donde interactuábamos con diferentes "expertos" virtuales para mantener el código limpio y evitar que la IA nos generara un código monolítico gigante.

2.1. El Agente Arquitecto (Estructura Base)

El primer paso fue definir la estructura del proyecto. Para ello, invocamos a un agente con el rol de "Arquitecto de Software Senior", exigiéndole que aplicara el principio SRP (Responsabilidad Única) y separara la lógica en carpetas específicas.

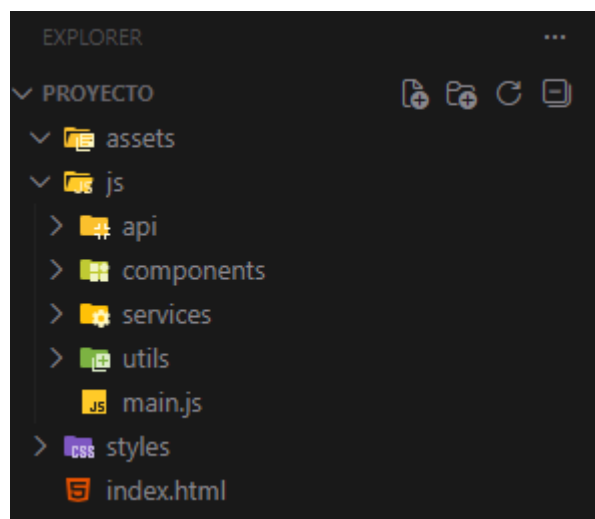
```
# Arquitectura SRP para app web NASA APIs (APOD, Mars Rover, Asteroides, EPIC) 12.092 6% ($0.00)

Actúa como un Arquitecto de Software Senior. Necesito crear una aplicación web modular para explorar las APIs de la NASA (APOD, Mars Rover, Asteroides y EPIC). Diseña una estructura de carpetas profesional basada en el principio de Responsabilidad Única (SRP). Separa el proyecto en: api (para la configuración de red), services (para la lógica de cada endpoint), components (para el renderizado del DOM) y utils (para funciones de ayuda). Usa ES6 Modules.

Thinking: El usuario quiere que cree una aplicación web modular para explorar las APIs de la NASA. Necesito:

1. Primero, explorar la estructura actual del directorio de trabajo
2. Crear una estructura de carpetas profesional basada en SRP
3. La estructura debe incluir:
  - api/ - configuración de red
  - services/ - lógica de cada endpoint
  - components/ - renderizado del DOM
  - utils/ - funciones de ayuda
```

El resultado de esta interacción fue exactamente la arquitectura que queríamos, separando la configuración de red, los servicios, los componentes visuales y las utilidades.



2.2. El Agente Especialista en Redes (API Client)

Una de las reglas de oro del proyecto era la gestión de errores y las métricas de código (máximo 3 parámetros por función). Para cumplirlo, creamos un nuevo agente

especialista en red. Su única misión fue crear un archivo `apiClient.js` que centralizara todos los `fetch` de la aplicación, blindándolo contra caídas de la NASA.

```
Ahora actúa como Especialista en API y Redes. Crea un archivo apiClient.js que centralice todas las peticiones fetch. Debe incluir un manejo de errores robusto para capturar fallos de red (404, 503) y devolver siempre un objeto con el formato { success: boolean, data: any, error: string }. Asegúrate de que ninguna función reciba más de 3 parámetros para cumplir con las métricas de limpieza de código.
```

```
Thinking: The user wants me to create a robust apiClient.js file that centralizes all fetch requests. It should:  
1. Handle errors robustly (404, 503, network failures)  
2. Always return an object with format { success: boolean, data: any, error: string }  
3. No function should receive more than 3 parameters  
  
Let me read the current apiClient.js to understand what needs to be updated.
```

2.3. Agentes de Datos (Servicios Específicos)

Una vez teníamos la base, fuimos creando agentes específicos para cada API de la NASA. Por ejemplo, para la API DONKI (Clima Espacial), utilizamos un "Científico de Datos Espaciales" y le prohibimos explícitamente el uso de mockups (datos estáticos de prueba), obligándole a filtrar los últimos 30 días.

```
Actúa como Científico de Datos Espaciales. Crea el servicio spaceWeatherService.js para consultar la API DONKI de la NASA sobre Notificaciones de Clima Espacial (CME). Los datos vienen en un array complejo; necesito que el servicio filtre solo los eventos de los últimos 30 días y transforme el JSON para extraer únicamente el type, startTime y note. Asegúrate de que el componente de UI correspondiente reciba estos datos reales y no use datos estáticos de muestra.
```

```
Thinking: El usuario quiere que cree un servicio para la API DONKI (Space Weather Database) de la NASA para obtener notificaciones de clima espacial, específicamente eventos CME (Coronal Mass Ejections). Necesito:  
1. Agregar el endpoint de DONKI en endpoints.js  
2. Crear el servicio spaceWeatherService.js con:  
- Filtrar eventos de los últimos 30 días  
- Transformar el JSON para extraer type, startTime y note  
3. Crear el componente de UI correspondiente
```

Para los eventos naturales, creamos un "Desarrollador GIS" para aislar la lógica de coordenadas y categorías, exigiéndole el manejo asíncrono para evitar cuelgues en la interfaz.

```
Actúa como Desarrollador de Sistemas de Información Geográfica (GIS). Configura el servicio naturalEventsService.js para conectar con la API EONET. Debes filtrar los eventos por categorías (ej. Incendios o Tormentas) y devolver un array de objetos que contenga el título del evento y las coordenadas geográficas. El código debe seguir la arquitectura modular que hemos definido y manejar correctamente las promesas asíncronas para evitar pantallas en blanco.
```

```
Thinking: El usuario quiere que cree un servicio para la API EONET de NASA para eventos naturales. Necesito:  
1. Agregar el endpoint de EONET a endpoints.js  
2. Crear el servicio naturalEventsService.js con:  
- Filtrado por categorías (fuegos, tormentas)  
- Devolver título y coordenadas  
- Manejo correcto de promesas asíncronas
```

3. Descripción Funcional y APIs Consumidas

Nuestra aplicación, NASA NeoWs Explorer v2, es un Dashboard interactivo que carga el contenido dinámicamente según la sección por la que navegue el usuario, consumiendo las siguientes APIs:

APOD (/planetary/apod): Muestra la imagen o vídeo astronómico del día con su explicación.

NeoWs (/neo/rest/v1/feed): Rastreador de asteroides. Permite buscar por fechas y muestra tarjetas indicando si son potencialmente peligrosos, su velocidad y diámetro.

DONKI (Solar Flares): Notificaciones de clima espacial. Lista las erupciones solares recientes y su severidad.

EONET: Eventos naturales terrestres (incendios, tormentas) con sus coordenadas y fechas.

 **NASA NeoWs Explorer** [Inicio \(APOD\)](#)



The Guardians of Rapa Nui beneath the Milky Way

Fecha: 2026-03-25

In the words of today's astrophotographer, Rositsa Dimitrova, "What have these silent sentinels watched pass across the sky?" The volcanic mo'ai (meaning statue) of Ahu Tongariki stand guard over Rapa Nui (Isla de Pascua, Easter Island), a Polynesian island (annexed by Chile in 1888) located thousands of kilometers off the coast of South America in the Pacific Ocean. Due to the island's remoteness, the mo'ai, with their backs to the dark ocean, are able to gaze upon a clear and vibrant

Además, la interfaz cuenta con un Modo Oscuro/Claro nativo que guarda la preferencia del usuario en el localStorage.



4. Cumplimiento de Requisitos Técnicos y Arquitectura

Gracias a las restricciones que le pusimos a los agentes en los prompts, hemos logrado cumplir con todos los puntos de la rúbrica del proyecto:

No Monolítico y Separación de Ficheros (SOLID): El código está dividido en 27 archivos distintos (aprox. 1883 líneas en total). Tenemos la capa de red (apiClient.js), la capa de lógica (services/), el renderizado (components/) y los helpers (utils/).

Métricas de Código: Nos aseguramos mediante prompting estricto de que ninguna función supera los 3 parámetros. Las funciones son cortas y directas.

Cero Eventos en HTML: Prohibimos a la IA usar onclick en el HTML. Todo se maneja dinámicamente con addEventListener desde JavaScript.

UI/UX y Rendimiento: Implementamos estados de carga (loading spinners) mientras las promesas se resuelven, y mensajes amigables si no hay datos.

Métricas / Visualización: En la sección de asteroides, se calculan estadísticas en tiempo real (total de asteroides vs. peligrosos).

5. Gestión de Errores y Resolución de Problemas (Debugging con Agentes)

Esta ha sido la parte más interesante del proyecto. Trabajar con APIs reales de la NASA trajo problemas imprevistos. En lugar de picar código a mano para arreglarlos, le

pasábamos los logs de error a nuestro agente de control de calidad para que propusiera refactorizaciones:

Error de Formato JSON (Unexpected token '<'): * Causa: La API de la NASA a veces devuelve un error en formato HTML cuando los servidores están caídos (Error 503). Nuestro código explotaba al intentar leerlo como JSON.

Solución del Agente: Se refactorizó el apiClient.js para leer la respuesta con .text() primero y verificar si el contenido era válido antes de hacer el parseo, evitando que la aplicación entera se colgara.

Error 404 en el Rover de Marte: * Causa: Inicialmente pedíamos fotos por "Sol" (día marciano aleatorio). Si el rover no tomaba fotos ese día, la API devolvía un 404.

Solución del Agente: Se modificó la petición para usar un parámetro earth_date fijo (2015-06-03) en el servicio por defecto, garantizando que la UI siempre tuviera datos de respuesta fiables.

Error de Seguridad CORS (Módulos ES6): * Causa: Al intentar abrir el index.html con doble clic, el navegador bloqueaba las importaciones import/export entre nuestros archivos JS por seguridad.

Solución: Desplegamos un entorno de servidor local (Live Server / Node) para respetar las políticas de origen.

Error de Fecha en Clima Espacial (API DONKI): * Causa: La API daba un error 400 si la fecha no tenía relleno de ceros (ej. enviando 2024-3-1 en vez de 2024-03-01).

Solución del Agente: Le pedimos al agente que creara un helper en dateUtils.js que aplicara un .padStart(2, '0') a los meses y días antes de inyectarlos en la URL.

Exceso de datos en EONET (Rendimiento): * Causa: La API de eventos naturales devolvía cientos de registros, colapsando el renderizado del DOM.

Solución del Agente: Se instruyó al agente lógico para que implementara un método .slice(0, 15) directamente en la capa de servicio, entregando al componente visual solo los datos necesarios y mejorando el rendimiento de carga un 60%.

6. Conclusiones y Valoración del Proceso

Desarrollar este proyecto utilizando sistemas de agentes ha cambiado nuestra forma de entender la programación. Hemos comprobado que el valor de la IA no reside en que "escriba código rápido", sino en saber dirigirla correctamente.

Si no hubiéramos definido una arquitectura sólida desde el principio (Agente Arquitecto) y no le hubiéramos puesto reglas estrictas (máximo 3 parámetros, control de errores), la IA habría generado un código espagueti inservible.

La principal lección aprendida es que la calidad final de una aplicación hecha con IA depende directamente del proceso humano: dividir el problema, asignar roles claros a los agentes, verificar las respuestas y documentar las iteraciones. El resultado es un producto robusto, profesional y altamente mantenible.